QUANTUM COMPUTING SERIES

The CHP Algorithm

Efficient Classical Simulation of Clifford Circuits

A conceptual guide to the ideas behind Aaronson & Gottesman's canonical stabilizer simulator — how it works, why it is fast, and what it can and cannot do

1. What Problem Does CHP Solve?

Quantum computing research often requires simulating quantum circuits on a classical computer — for testing algorithms, verifying hardware behavior, or benchmarking compilers. But the general simulation problem is exponentially hard: a circuit on n qubits requires tracking 2^n complex numbers, and that number doubles with every qubit you add.

However, there is a restricted but important class of quantum circuits — Clifford circuits — that turn out to be classically simulable in polynomial time. The CHP algorithm, introduced by Scott Aaronson and Daniel Gottesman in 2004, is the canonical implementation of this idea. The name stands for the three fundamental gate types it handles: CNOT, Hadamard (H), and Phase (S).

The Key Question CHP Answers

Given a quantum circuit made entirely of H, S, and CNOT gates, followed by Pauli measurements — can we simulate it efficiently? CHP says yes: it tracks the state using $O(n^2)$ bits of memory and processes each gate in $O(n)$ time, each measurement in $O(n^2)$ time.

2. The Starting Point: Stabilizer States

CHP works by exploiting the algebraic structure of a special class of quantum states called stabilizer states. Rather than storing the full wavefunction — all 2^n complex amplitudes — CHP stores a compact description of the state in terms of the Pauli operators that leave it invariant.

Every stabilizer state can be uniquely identified by a group of n commuting Pauli operators called its stabilizers. Each stabilizer P satisfies $P|\psi\rangle = |\psi\rangle$ — it is a symmetry of the state. Specifying n such operators is enough to pin down the state exactly, and each operator can be encoded in a binary string of length 2n+1. The total description takes $O(n^2)$ bits.

Why This Is Useful

A general n-qubit state needs 2^n complex numbers. A stabilizer state needs only n binary strings. For 1000 qubits, the difference is between a number requiring more storage than all data on Earth versus a megabyte. The catch is that stabilizer states are a restricted

subset of all quantum states — but they include many practically important ones, like Bell states, GHZ states, cluster states, and quantum error-correcting codes.

3. The Tableau: CHP's Data Structure

CHP stores the state in a rectangular binary table called the stabilizer tableau. For n qubits, this is a matrix with 2n rows and 2n+1 columns.

3.1 What the Columns Represent

Each row of the tableau encodes a single Pauli generator. The 2n+1 columns are divided as follows:

- Columns 0 to n-1: The X component of each qubit in this generator (1 if the qubit has X or Y, 0 otherwise).
- Columns n to 2n-1: The Z component of each qubit (1 if the qubit has Z or Y, 0 otherwise).
- Column 2n: A sign bit (0 means positive, 1 means negative phase).

The combination (X=1, Z=1) encodes the Y Pauli. This binary encoding is elegant: it turns the abstract algebra of Pauli operators into bitwise arithmetic.

3.2 Two Types of Generators: Stabilizers and Destabilizers

The key innovation of the 2004 paper is storing two sets of generators, not just one. The tableau's 2n rows are split:

- The top n rows hold destabilizer generators — mathematical companions to the stabilizers.
- The bottom n rows hold the stabilizer generators themselves.

▶ **The Role of Destabilizers**

Gottesman's 1997 formulation stored only stabilizers. This was sufficient to describe the state and apply gates, but deterministic measurement outcomes required searching through all 2^n group elements — exponential cost. Aaronson and Gottesman's key contribution was introducing destabilizers: a paired set of operators that form a symplectic basis with the stabilizers. With destabilizers, deterministic measurements become an $O(n^2)$ row operation instead of an exponential search. This is the main algorithmic contribution of the CHP paper.

The destabilizer of generator g_i is a Pauli operator that anticommutes with g_i but commutes with all other stabilizers and destabilizers. Together, the paired structure forms a clean mathematical basis that makes the measurement algorithm work.

4. Gate Updates: Rewriting Generators

When a Clifford gate is applied to the state, the stabilizer generators transform by conjugation: each generator g becomes UgU†, where U is the gate. Because Clifford gates map Paulis to Paulis, the new generator is still a Pauli — it just has a different X/Z pattern and possibly a different sign.

For each of the three fundamental gates, the update rules are exact and purely bitwise. Here is what happens to the tableau for each gate:

4.1 The Hadamard Gate (H) on Qubit a

H swaps X and Z components at position a. In the Pauli world: X maps to Z, Z maps to X, and Y maps to -Y (a sign flip). In the tableau: swap the X bit and Z bit at column a for every row. If both are 1 (meaning the local Pauli was Y), flip the sign bit as well.

4.2 The Phase Gate (S) on Qubit a

S maps X to Y and leaves Z unchanged. In the tableau: add the X bit to the Z bit at column a for every row (so X becomes XZ = Y). If the local Pauli was already Y, flip the sign bit because S maps Y to -X.

4.3 The CNOT Gate (Control a, Target b)

CNOT has a more complex action. Its key effects: X on the control qubit spreads to the target (X⊗I becomes X⊗X), and Z on the target qubit spreads back to the control (I⊗Z becomes Z⊗Z). In the tableau: XOR the X bit of the target with the X bit of the control, and XOR the Z bit of the control with the Z bit of the target. The sign update requires a small phase correction accounting for cases where the spreading creates an implicit Y.

> **Efficiency**
> Each gate loop touches all 2n rows of the tableau, and each row takes O(1) work (a few bitwise operations). Total cost per gate: O(n). Applying a depth-d circuit to n qubits costs O(nd) time — polynomial in everything.

5. The rowmul Operation: Multiplying Two Generators

Both the gate update logic and the measurement algorithm rely on a core helper operation called rowmul, which multiplies two Pauli generators and updates one of them in place.

The binary part of the multiplication — the X and Z components — is simple: XOR the corresponding bits. When you multiply two Paulis, the result's X part is the XOR of their X parts, and similarly for Z.

The hard part is the sign. Pauli multiplication generates phases. At each qubit position, multiplying two single-qubit Paulis can contribute a factor of i, -i, -1, or 1 depending on the combination. For example:

- X times Y gives iZ — the phase is i.
- Z times X gives iY — the phase is i.
- Y times Y gives -I — the phase is -1.

CHP sums these phase contributions across all n qubit positions (working modulo 4 to track powers of i), then combines with the existing sign bits of both rows to get the final sign of the product. Only phases of +1 or -1 should appear in valid stabilizer generators — if a factor of ±i emerges, something has gone wrong.

> **Why This Matters**
> The sign computation in rowmul is the most common source of bugs in CHP implementations. Getting the phase wrong produces no error message — the tableau just quietly becomes inconsistent, and future measurements give wrong answers. The careful phase tracking using powers-of-i arithmetic is what makes the algorithm correct, not just fast.

6. The Measurement Algorithm

Measuring qubit a in the computational basis is equivalent to measuring the Pauli operator Z on that qubit. CHP handles this in two distinct cases depending on whether the outcome is predetermined by the state.

6.1 Case 1: Random Outcome

This case applies when at least one stabilizer generator anticommutes with the Z measurement. Anticommuting means the generator has an X or Y component on qubit a — which is precisely when that generator does not commute with Z.

This case corresponds to the state being in a genuine superposition with respect to the measurement: the outcome is 50/50 random. Here is the procedure:

- Find one anticommuting stabilizer row as a pivot.
- For every other row (both stabilizers and destabilizers) that also anticommutes with Z on qubit a, multiply that row by the pivot row. This clears the anticommuting structure from all other rows.
- Copy the old pivot stabilizer into the corresponding destabilizer slot, preserving the symplectic pairing.
- Replace the pivot stabilizer with the sampled outcome: either +Z or -Z on qubit a (with all other positions set to identity), with the sign bit reflecting the random result.

6.2 Case 2: Deterministic Outcome

This case applies when every stabilizer generator commutes with Z on qubit a — meaning the state is entirely in one eigenspace of Z. The outcome is fixed, not random.

To find the outcome without modifying the state: use the destabilizer rows as pointers. Scan the destabilizers; if destabilizer j has an X component on qubit a, multiply a scratch accumulator row by the corresponding stabilizer row n+j. After processing all relevant destabilizers, the scratch row equals ±Z on qubit a — and its sign bit is the answer. The tableau is not modified.

> **Why Two Cases?**
> The random case corresponds to a wavefunction collapse — the state actually changes after measurement. The deterministic case is just reading off information that was already encoded in the state. The destabilizer structure makes the deterministic case efficient by identifying exactly which subset of stabilizers needs to be multiplied together, rather than searching through all 2^n possibilities.

7. Complexity: Why It Scales Well

The efficiency of CHP can be summarized in a simple table:

Operation	Cost	Bottleneck
Gate H or S	O(n)	Updates all 2n rows, O(1) work each
Gate CNOT	O(n)	Updates all 2n rows, O(1) work each
Measurement (random)	$O(n^2)$	Up to n rowmul calls, each O(n)
Measurement (deterministic)	$O(n^2)$	Up to n scratch multiplications, each O(n)
Memory	$O(n^2)$ bits	2n rows × (2n+1) columns

Compare this to statevector simulation:

Scale	Memory Comparison
n = 10 qubits	Tableau: ~200 bits | Statevector: 16 KB
n = 100 qubits	Tableau: ~25 KB | Statevector: 10^{22} bytes (impossible)
n = 1000 qubits	Tableau: ~250 KB | Statevector: does not exist
n = 10,000 qubits	Tableau: ~200 MB | Statevector: impossible

The exponential advantage of CHP over statevector simulation grows with every qubit. This is why CHP-style simulators like Stim can handle millions of qubits in quantum error correction simulations — something no general-purpose quantum simulator could approach.

8. Measuring Arbitrary Pauli Observables

The basic measurement operation handles Z on a single qubit. To measure an arbitrary multi-qubit Pauli observable M — say, X on qubit 1 and Z on qubit 3 and Y on qubit 5 — CHP uses a standard reduction strategy.

The idea: any Pauli M can be transformed into a Z on a single qubit by applying a sequence of Clifford gates. Specifically:

- X on a qubit becomes Z after applying H to that qubit.
- Y on a qubit becomes Z after applying S† and then H.
- Z is already Z, no action needed.
- I (identity) means that qubit is not involved.

After these single-qubit rotations, the observable becomes a tensor product of Zs on some subset of qubits. A cascade of CNOT gates can then compress this multi-qubit Z observable into a single Z on one anchor qubit. Measure that qubit, then undo all the gates in reverse order to restore the original frame.

This works because the entire rotation — including the gates and their undoing — is Clifford, so it can all be done within the tableau framework.

9. What CHP Cannot Do

CHP is powerful within its domain, but it has hard limits:

- No T gates. The T gate (which adds a phase of e^(iπ/4) to the |1⟩ component) is not Clifford. Applying T to a stabilizer state produces a non-stabilizer state — a state that cannot be described by any Pauli stabilizer group. CHP cannot track this.
- No amplitude access. CHP knows everything about the stabilizer structure but cannot directly output the probability amplitudes of individual basis states. Computing such amplitudes from the tableau requires additional work and is not always $O(n^2)$.
- No non-Pauli measurements. Measuring in a basis defined by a non-Pauli observable is outside the framework.

> **Extending Beyond CHP: Clifford + T**
> If a circuit contains t T gates in addition to Clifford operations, simulation can still be done by expressing the state as a weighted sum of stabilizer states — one for each T gate

injection. The cost scales roughly as $2^t \times O(n^2)$: still polynomial in qubits, but exponential in the number of T gates. This is the foundation of several advanced simulation strategies and makes T-count the central cost metric in fault-tolerant quantum computing.

10. Applications of CHP

10.1 Quantum Error Correction Simulation

This is CHP's dominant practical application. Quantum error correction codes — repetition codes, surface codes, toric codes, color codes — are all stabilizer codes. Their entire lifecycle of syndrome extraction, error injection, decoding, and correction involves only Clifford operations and Pauli measurements. CHP-style simulators like Stim can simulate these workflows at millions-of-qubit scale, making them indispensable for designing and benchmarking fault-tolerant architectures.

10.2 Randomized Benchmarking

Randomized benchmarking (RB) is a standard protocol for measuring average gate fidelity in quantum hardware. It applies long random sequences of Clifford gates, then inverts the sequence and measures whether the result is close to the identity. The ideal (noise-free) behavior is modeled using CHP, providing the reference against which hardware noise is measured.

10.3 Clifford Circuit Compilation

Any Clifford unitary can be compiled to the {H, S, CNOT} gate set. Verifying that a compilation is correct — that the output circuit implements the same unitary as the input — can be done by running both through CHP on test inputs and checking that the stabilizer generators match. This is much faster than full matrix comparison.

10.4 Formal Verification and Testing

For algorithms where the classical part of the computation produces a Clifford + measurement circuit, CHP provides a fast independent oracle. Running thousands of random test cases through both a high-level implementation and a CHP reference is an efficient way to catch correctness bugs.

11. A Visual Summary: CHP in One View

Component	Description
State representation	2n × (2n+1) binary tableau — destabilizers (rows 0 to n-1) and stabilizers (rows n to 2n-1)
Gate H on qubit a	Swap X and Z bits at column a across all rows; flip sign where both were 1 (Y)

Component	Description
Gate S on qubit a	XOR Z bit with X bit at column a; flip sign where both are 1 (Y)
Gate CNOT (a→b)	XOR X of target with X of control; XOR Z of control with Z of target; apply sign correction
rowmul (i × k)	XOR x and z blocks; accumulate per-qubit phase contributions mod 4 to update sign
Measurement (random)	Find anticommuting pivot; clear all other anticommuting rows; replace pivot with outcome ±Z
Measurement (deterministic)	Scan destabilizers; accumulate a scratch product of stabilizers; read off sign
Gate cost	O(n) per gate — update all 2n rows
Measurement cost	$O(n^2)$ — up to n row multiplications
Memory	$O(n^2)$ bits total — dramatically better than 2^n complex amplitudes
Limitation	Clifford gates only — T gates and other non-Clifford operations are outside the framework

12. Historical Context and Further Reading

Stabilizer states were first studied by Gottesman in his 1997 PhD thesis at Caltech, in the context of quantum error correction. The original formalism stored only stabilizer generators, but deterministic measurements required exponential search.

The key algorithmic improvement — the destabilizer structure that makes deterministic measurements efficient — was introduced by Aaronson and Gottesman in their 2004 Physical Review A paper, which also gave the algorithm its name and proved the $O(n^2)$ time bounds rigorously.

In 2021, Craig Gidney released Stim, a highly optimized implementation of the CHP framework that exploits SIMD (single instruction, multiple data) instructions on modern CPUs to process 256 tableau rows simultaneously. Stim can handle circuits with billions of gates and simulate quantum error correction at the scale needed for practical fault-tolerant quantum computing.

- Aaronson & Gottesman (2004) — Improved Simulation of Stabilizer Circuits. Physical Review A 70, 052328. The primary CHP paper.
- Gottesman (1997) — Stabilizer Codes and Quantum Error Correction. PhD thesis, Caltech. The original stabilizer formalism.
- Gidney (2021) — Stim: A Fast Stabilizer Circuit Simulator. Quantum 5, 497. State-of-the-art implementation.
- Anders & Briegel (2006) — Fast simulation of stabilizer circuits using a graph-state representation. Physical Review A 73, 022334.
- Bravyi & Gosset (2016) — Improved classical simulation of quantum circuits dominated by linear optical elements. Physical Review Letters 116. Extends CHP to Clifford+T circuits.